

Similarity Measures

Instructor: Ayesha Enayet

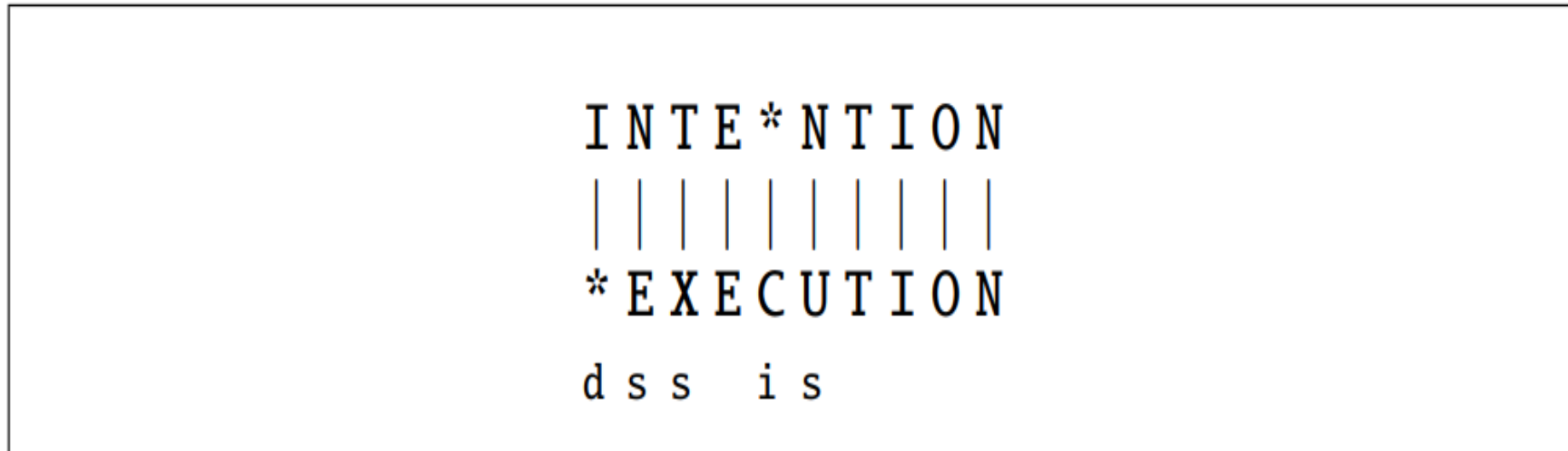


Figure 2.13 Representing the minimum **edit distance** between two strings as an **alignment**. The final row gives the operation list for converting the top string into the bottom string: d for deletion, s for substitution, i for insertion.

Formal definition (Dynamic Programming)

- The minimum edit distance between two strings. Given two strings, the source string X of length n , and target string Y of length m , we'll define $D(i, j)$ as the edit distance between $X[1..i]$ and $Y[1..j]$, i.e., the first i characters of X and the first j characters of Y . The edit distance between X and Y is thus $D(n, m)$.

[illegible]

$$D[i, j] = \min \begin{cases} D[i-1, j] + \text{del-cost}(\text{source}[i]) \\ D[i, j-1] + \text{ins-cost}(\text{target}[j]) \\ D[i-1, j-1] + \text{sub-cost}(\text{source}[i], \text{target}[j]) \end{cases}$$

If we assume the version of Levenshtein distance in which the insertions and deletions each have a cost of 1 ($\text{ins-cost}(\cdot) = \text{del-cost}(\cdot) = 1$), and substitutions have a cost of 2 (except substitution of identical letters have zero cost), the computation for $D(i, j)$ becomes:

$$D[i, j] = \min \begin{cases} D[i-1, j] + 1 \\ D[i, j-1] + 1 \\ D[i-1, j-1] + \begin{cases} 2; & \text{if } \text{source}[i] \neq \text{target}[j] \\ 0; & \text{if } \text{source}[i] = \text{target}[j] \end{cases} \end{cases} \quad (2.4)$$

Src\Tar	#	e	x	e	c	u	t	i	o	n
#	0	1	2	3	4	5	6	7	8	9
i	1	2	3	4	5	6	7	6	7	8
n	2	3	4	5	6	7	8	7	8	7
t	3	4	5	6	7	8	7	8	9	8
e	4	3	4	5	6	7	8	9	10	9
n	5	4	5	6	7	8	9	10	11	10
t	6	5	6	7	8	9	8	9	10	11
i	7	6	7	8	9	10	9	8	9	10
o	8	7	8	9	10	11	10	9	8	9
n	9	8	9	10	11	12	11	10	9	8

Figure 2.17 Computation of minimum **edit distance** between *intention* and *execution* with the algorithm of Fig. 2.16, using Levenshtein distance with cost of 1 for insertions or deletions, 2 for substitutions.

	#	e	x	e	c	u	t	i	o	n
#	0	← 1	← 2	← 3	← 4	← 5	← 6	← 7	← 8	← 9
i	↑ 1	↖←↑ 2	↖←↑ 3	↖←↑ 4	↖←↑ 5	↖←↑ 6	↖←↑ 7	↖ 6	← 7	← 8
n	↑ 2	↖←↑ 3	↖←↑ 4	↖←↑ 5	↖←↑ 6	↖←↑ 7	↖←↑ 8	↑ 7	↖←↑ 8	↖ 7
t	↑ 3	↖←↑ 4	↖←↑ 5	↖←↑ 6	↖←↑ 7	↖←↑ 8	↖ 7	←↑ 8	↖←↑ 9	↑ 8
e	↑ 4	↖ 3	← 4	↖← 5	← 6	← 7	←↑ 8	↖←↑ 9	↖←↑ 10	↑ 9
n	↑ 5	↑ 4	↖←↑ 5	↖←↑ 6	↖←↑ 7	↖←↑ 8	↖←↑ 9	↖←↑ 10	↖←↑ 11	↖↑ 10
t	↑ 6	↑ 5	↖←↑ 6	↖←↑ 7	↖←↑ 8	↖←↑ 9	↖ 8	← 9	← 10	←↑ 11
i	↑ 7	↑ 6	↖←↑ 7	↖←↑ 8	↖←↑ 9	↖←↑ 10	↑ 9	↖ 8	← 9	← 10
o	↑ 8	↑ 7	↖←↑ 8	↖←↑ 9	↖←↑ 10	↖←↑ 11	↑ 10	↑ 9	↖ 8	← 9
n	↑ 9	↑ 8	↖←↑ 9	↖←↑ 10	↖←↑ 11	↖←↑ 12	↑ 11	↑ 10	↑ 9	↖ 8

Exercise

Src\Tar	#	a	b	c
#				
b				
b				
a				

Exercise

Src\Tar	#	a	b	c
#	0	1	2	3
b	1	2	1	2
b	2	3	2	3
a	3	2	3	4

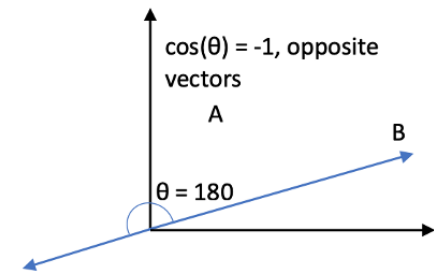
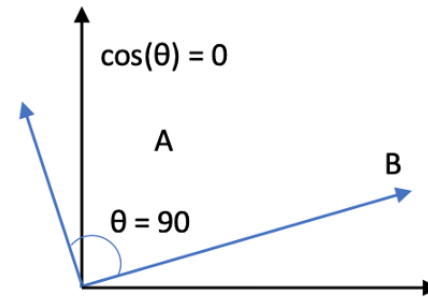
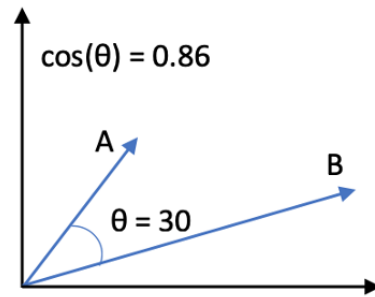
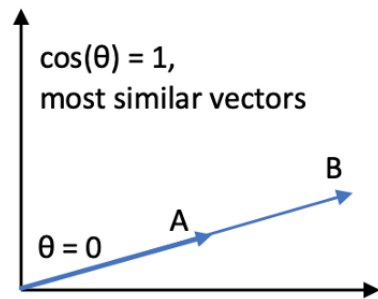
Hamming distance

- Hamming Distance refers to the number of positions at which two strings of the same length differ. It is a metric used in computer science to measure dissimilarity between strings.
- e.g. the hamming distance between aba and abb is 1.
- Could be used to measure a difference between two sequences

Cosine Similarity

- Taken from <https://towardsdatascience.com/cosine-similarity-how-does-it-measure-similarity-50ad30aad7d1>

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$



- $X=\{3,2,0,5\}$
- $Y=\{1,0,0,0\}$

$$x \cdot y = 3*1 + 2*0 + 0*0 + 5*0 = 3$$

$$\|x\| = \sqrt{(3)^2 + (2)^2 + (0)^2 + (5)^2} = 6.16$$

$$\|y\| = \sqrt{(1)^2 + (0)^2 + (0)^2 + (0)^2} = 1$$

$$\therefore S_C(x, y) = 3 / (6.16 * 1) = 0.49$$

Taken from <https://www.geeksforgeeks.org/dbms/cosine-similarity/>

Cosine Similarity

- Find the cosine similarity between the following vectors:
- $A=[1,2,3]$
- $B=[4,5,6]$